

A Comprehensive Measure of Well-Being (by Using Machine Learning)

Project Description:

A Comprehensive Measure of Well-Being is a web-based application designed to assess and monitor an individual's overall well-being. The system collects information related to physical health, mental health, emotional well-being, lifestyle, sleep, stress, and daily habits through questionnaires and user inputs. Based on the collected data, the application analyzes the user's well-being and provides a well-being score along with personalized recommendations to improve their health and quality of life.

The application offers an easy-to-use interface where users can register, log in securely, complete assessments, and track their progress over time. It also includes data visualization through charts and graphs, allowing users to understand changes in their well-being and identify areas that need improvement.

Problem Statement

In today's fast-paced lifestyle, people often focus on only one aspect of health, such as physical fitness, while neglecting mental, emotional, and social well-being. As a result, stress, anxiety, unhealthy habits, and poor work-life balance have become increasingly common. Existing health applications typically track only specific parameters, such as fitness or diet, without providing a complete picture of an individual's overall well-being.

There is a need for a comprehensive system that can assess multiple dimensions of well-being, including physical health, mental health, emotional state, lifestyle, sleep, and daily habits. Such a system should analyze user data, generate an overall well-being score, and provide personalized recommendations to help users improve their quality of life and maintain a healthy lifestyle.

Scenarios:

Scenario 1: Daily Health Assessment

A senior citizen logs into the application and completes a daily questionnaire about sleep, physical activity, mood, stress, and health. The system calculates a well-being score and provides health recommendations.

Scenario 2: Health Monitoring

The application tracks the user's well-being over time. If the score decreases consistently, it alerts the user to seek medical advice or improve lifestyle habits.

Scenario 3: Personalized Recommendations

Based on the assessment results, the system suggests activities such as light exercise, meditation, a balanced diet, proper sleep, or increased social interaction to improve overall well-being.

Scenario 4: Caregiver Support

A caregiver or family member can review the senior citizen's well-being reports and monitor changes, enabling timely support when needed.

Scenario 5: Progress Tracking

The user can view charts and reports showing improvements or declines in their physical, mental, and emotional well-being over weeks or months, helping them maintain a healthier lifestyle.

Objectives:

- To assess the overall well-being of senior citizens using multiple health parameters.
- To monitor physical, mental, emotional, and social well-being in one platform.
- To calculate a comprehensive well-being score based on user responses.
- To provide personalized health and lifestyle recommendations.
- To help senior citizens adopt healthier daily habits.
- To enable caregivers and family members to monitor the well-being of seniors.
- To improve the quality of life through regular health assessment and progress tracking.

Scope

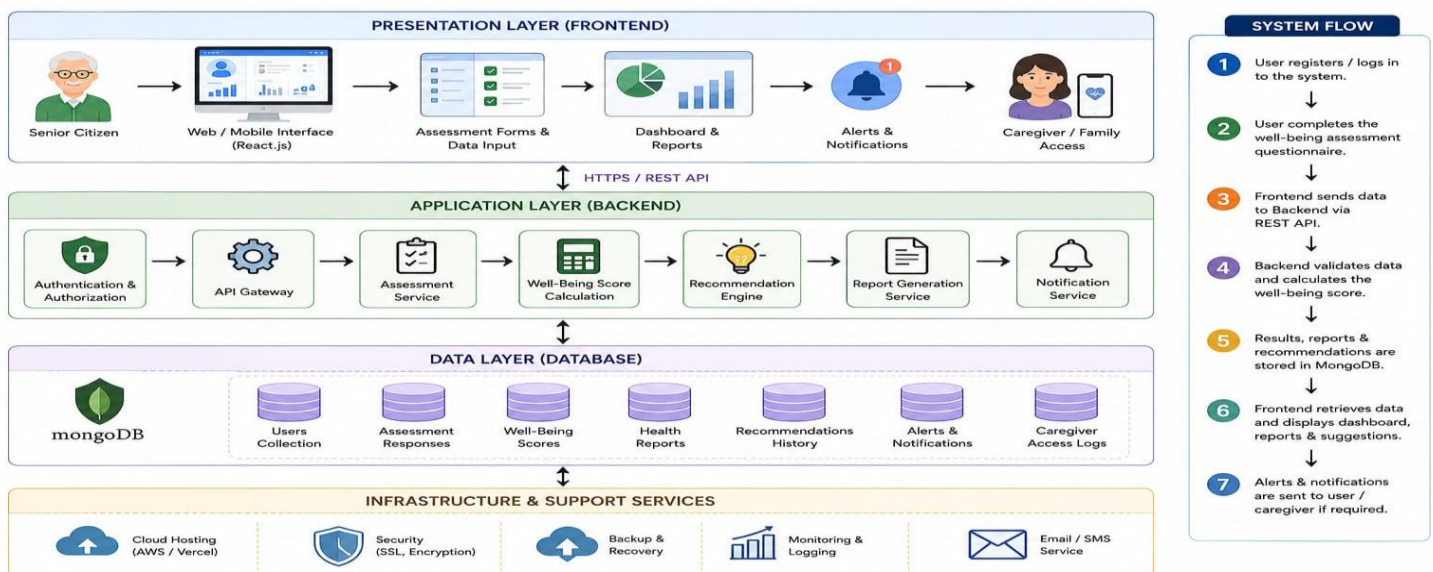
- Collects information related to physical health, mental health, emotional state, sleep, nutrition, and daily activities.
- Evaluates overall well-being using questionnaires and user inputs.
- Generates a well-being score and detailed health reports.
- Provides personalized recommendations for improving health and lifestyle.
- Tracks user progress over time with charts and reports.
- Supports caregivers by allowing them to monitor the health status of senior citizens.
- Can be used in homes, hospitals, senior care centers, and community health programs to promote healthy aging.

Technical Architecture:

Architecture Fl

TECHNICAL ARCHITECTURE: ARCHITECTURE FLOW

A Comprehensive Measure of Well-Being for Senior Citizens

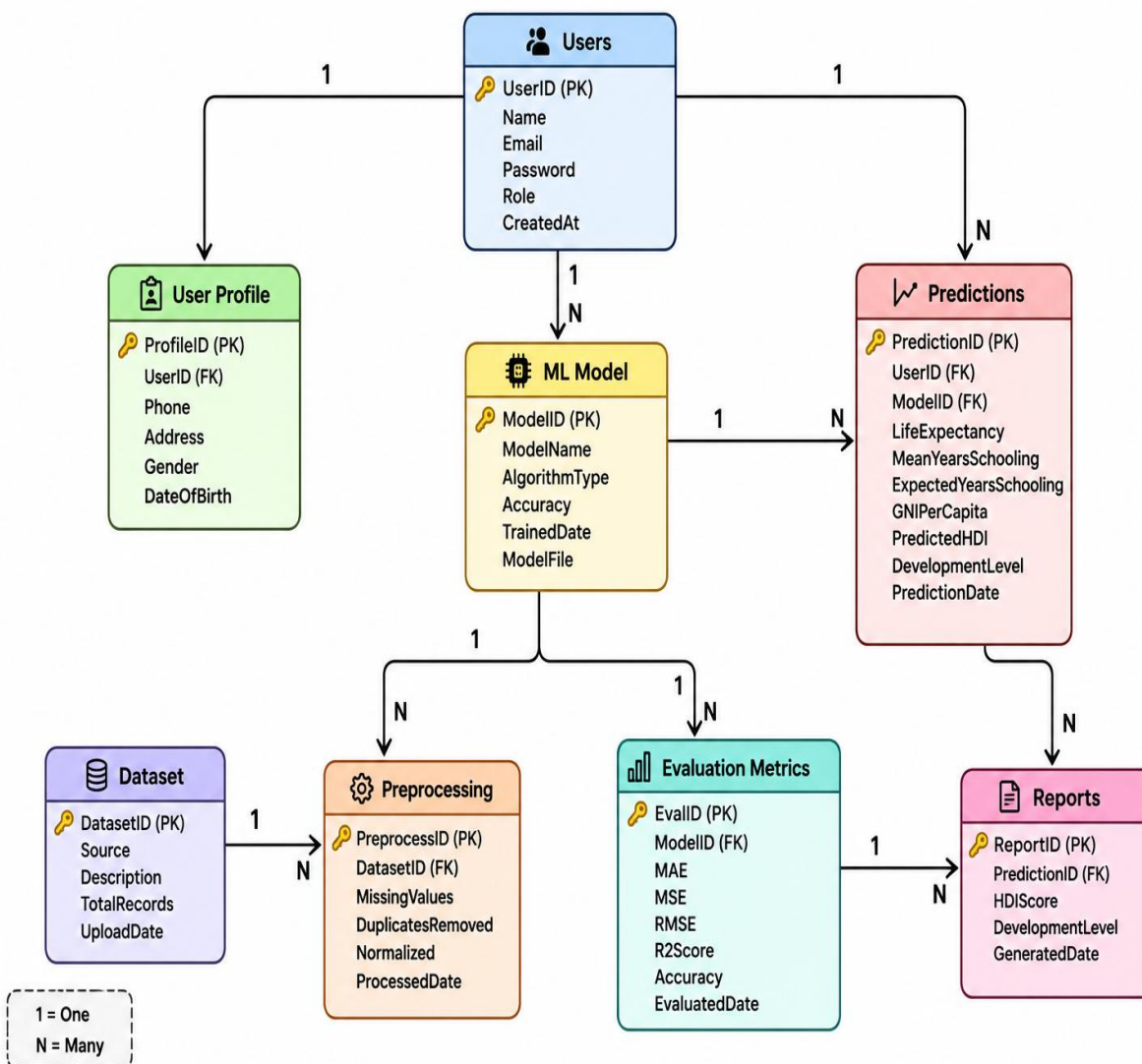


ERDiagram

Description:

The **Entity Relationship (ER) Diagram** represents the database structure of the **Comprehensive Measure of Well-Being for Senior Citizens** system. It illustrates how different entities are connected and how data flows within the application. The **User** entity stores the personal and login details of senior citizens and caregivers. Each user can register, log in, and access the system to complete well-being assessments.

The **Well-Being Assessment** entity records the responses provided by users regarding their physical health, mental health, emotional well-being, lifestyle, sleep quality, and daily activities. A user can complete multiple assessments over time to monitor changes in their health.



Pre-requisites

- **Python Programming Proficiency (Python 3.x)**
- **Machine Learning Fundamentals — scikit-learn**
- **Data Handling Libraries — Pandas, NumPy**
- **Data Visualization — Matplotlib / Seaborn (for EDA)**
- **Web Framework Knowledge — Flask, for deployment of the prediction interface**
- **Version Control with Git**

Project Workflow:

Milestone 1: Requirement Analysis

- Identify project objectives and user requirements.
- Define the scope of the well-being monitoring system.

Milestone 2: System Design

- Design the system architecture, ER diagram, and database schema.
- Create UI wireframes and workflow.

Milestone 3: Frontend Development

- Develop the landing page.
- Create Login and Registration pages.
- Develop the Dashboard and Assessment forms.

Milestone 4: Backend Development

- Build REST APIs using Node.js and Express.js.
- Implement user authentication and assessment processing.

Milestone 5: Database Development

- Create MongoDB collections.
- Store user details, assessment data, scores, and recommendations.

Milestone 6: Well-Being Score & Recommendation Module

- Calculate the overall well-being score.
- Generate personalized health recommendations.

Milestone 7: Testing

- Perform unit testing, integration testing, and fix bugs.
- Verify system functionality.

Milestone 8: Deployment & Documentation

- Deploy the application.
- Prepare project documentation, presentation, and final report.

Methodology

Dataset Description

- **Dataset:** HDI.csv
- **Missing Values:** Checked and handled
- **Duplicate Records:** Removed (if any)
- **Feature Selection:** 4 Numerical Features
- **Target Variable:** HDI
- **Train-Test Split:** 80% Training, 20% Testing
- **Preprocessing Libraries Used:** Pandas, NumPy, Scikit-learn

Data Preprocessing Steps

- Collected the **HDI.csv** dataset containing Human Development Index information.
- Loaded the dataset into Python using the **Pandas** library.
- Explored the dataset by checking its structure, dimensions, and data types.
- Generated statistical summaries to understand the distribution of the data.
- Checked for missing (null) values and handled them appropriately.
- Verified and removed duplicate records, if any.
- Selected the four input features:
 - Life Expectancy
 - Mean Years of Schooling
 - Expected Years of Schooling
 - Gross National Income (GNI) per Capita
- Selected **HDI** as the target variable for prediction.
- Split the dataset into **80% training data** and **20% testing data**.
- Validated the processed dataset to ensure it was clean and ready for model training.
- Used the preprocessed data to train the **Linear Regression** model.

Algorithms Used

-  Linear Regression
-  Train-Test Split (80:20)
-  Flask Web Application Deployment
-  Data Preprocessing

Evaluation Metrics

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R² Score (Coefficient of Determination)
- Prediction Accuracy
- Model Performance Analysis
- Actual vs Predicted Value Comparison
- Error Analysis

Web Application:

The project includes a **Flask-based web application** that provides a simple and interactive interface for predicting the Human Development Index (HDI). Users can enter the four required socio-economic indicators—**Life Expectancy, Mean Years of Schooling, Expected Years of Schooling, and Gross National Income (GNI) per Capita**—through an input form. The application validates the entered values, passes them to the trained **Linear Regression** model, and generates the predicted HDI score.

The predicted score is then classified into one of the four human development categories: **Low, Medium, High, or Very High Human Development**. The result page displays both the HDI score and its corresponding development tier in an easy-to-understand format. The application is built using **Python, Flask, HTML, CSS, and JavaScript**, providing a responsive and user-friendly interface for real-time HDI prediction.

Application Form

Human Development Index

Predict a country's HDI from four key indicators

Life Expectancy


82.6

Mean Years of Schooling


12.9

Expected Years of Schooling


18.2

GNI per Capita (US\$)


66000

Predict HDI →

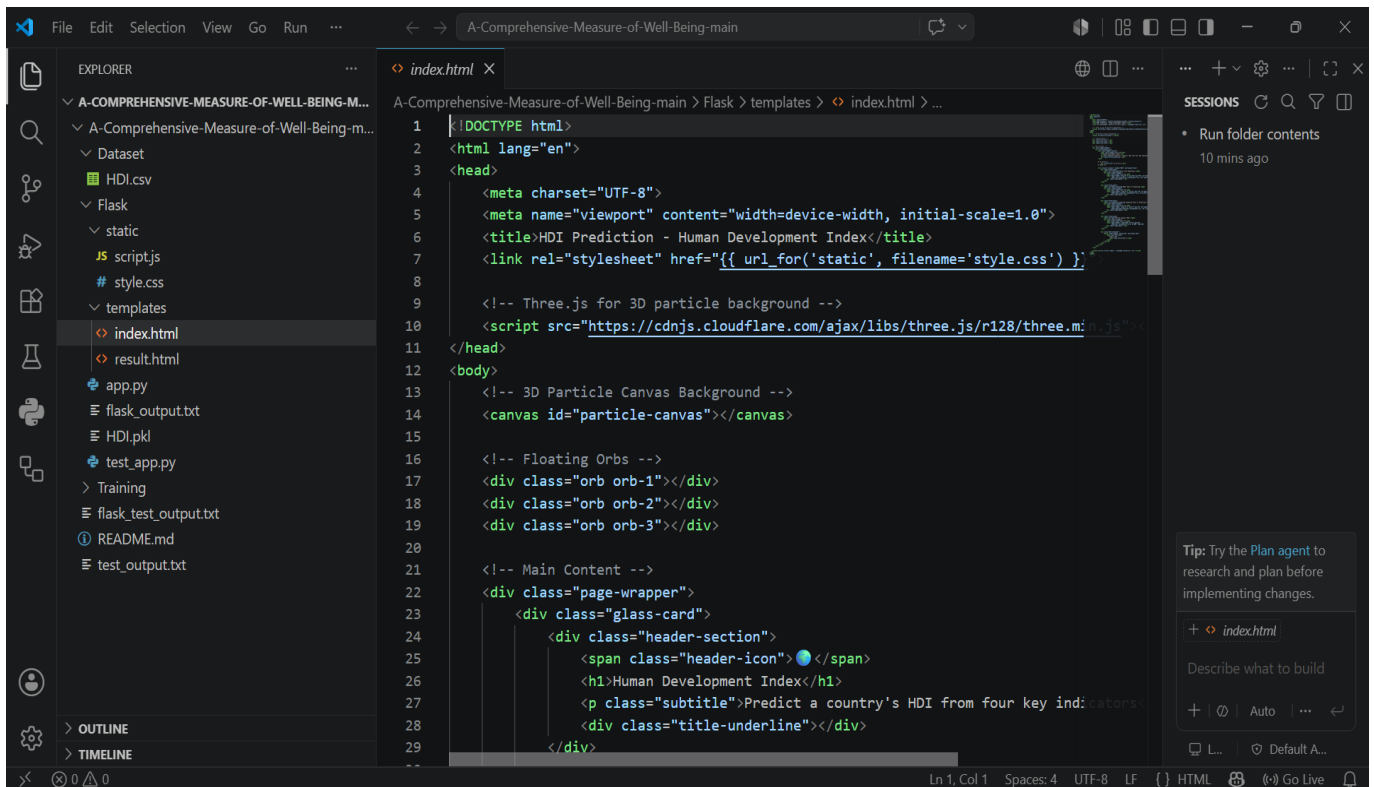
Prediction of Result



Frontend Implementation

- HTML
- CSS
- JavaScript

HTML(structure)



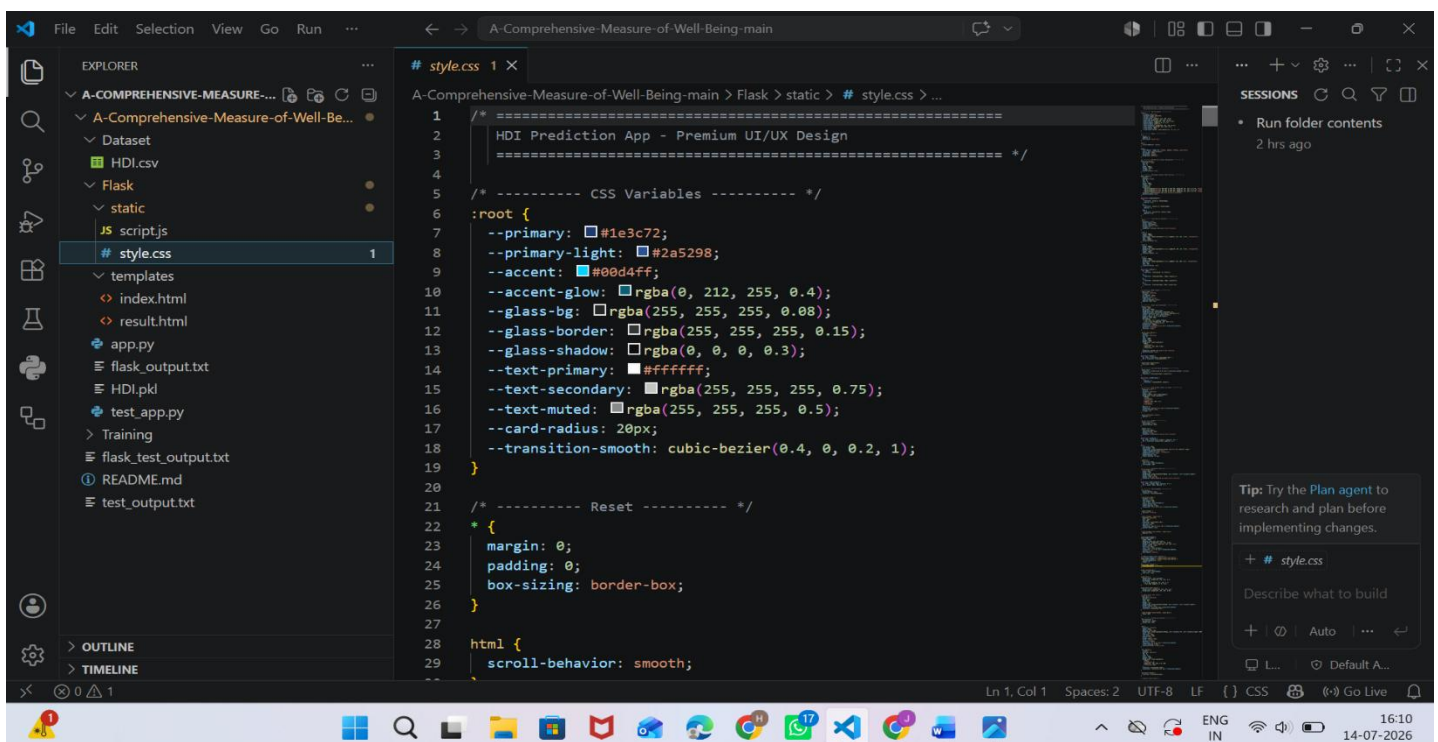
The screenshot shows the VS Code editor with the file explorer on the left displaying the project structure. The main editor window shows the content of `index.html`. The HTML structure includes a DOCTYPE declaration, a head section with meta tags for charset, viewport, and title, and a link to a stylesheet. The body section contains comments for 3D particle background and floating orbs, followed by a main content wrapper with a glass card containing a header section with an icon, title, and subtitle.

```

1 |<!DOCTYPE html>
2 |<html lang="en">
3 |<head>
4 |  <meta charset="UTF-8">
5 |  <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 |  <title>HDI Prediction - Human Development Index</title>
7 |  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
8 |
9 |  <!-- Three.js for 3D particle background -->
10 |  <script src="https://cdnjs.cloudflare.com/ajax/libs/three.js/r128/three.min.js"></script>
11 |</head>
12 |<body>
13 |  <!-- 3D Particle Canvas Background -->
14 |  <canvas id="particle-canvas"></canvas>
15 |
16 |  <!-- Floating Orbs -->
17 |  <div class="orb orb-1"></div>
18 |  <div class="orb orb-2"></div>
19 |  <div class="orb orb-3"></div>
20 |
21 |  <!-- Main Content -->
22 |  <div class="page-wrapper">
23 |    <div class="glass-card">
24 |      <div class="header-section">
25 |        <span class="header-icon"></span>
26 |        <h1>Human Development Index</h1>
27 |        <p class="subtitle">Predict a country's HDI from four key indicators</p>
28 |        <div class="title-underline"></div>
29 |      </div>

```

CSS



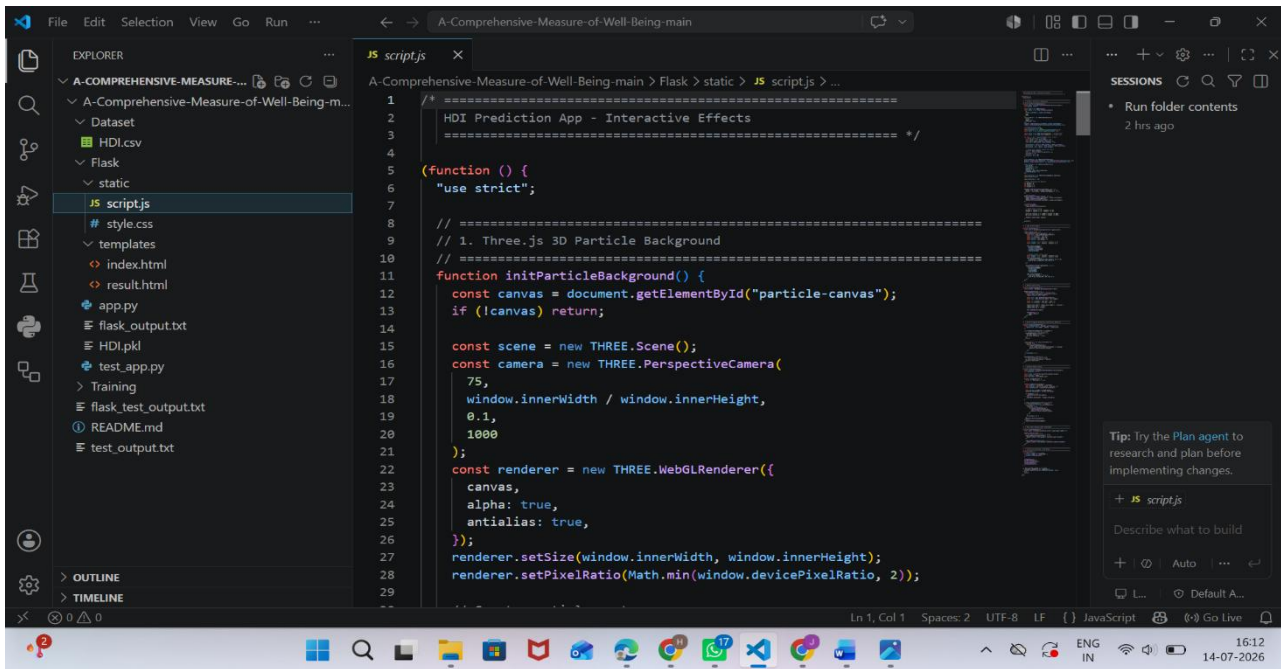
The screenshot shows the VS Code editor with the file explorer on the left displaying the project structure. The main editor window shows the content of `style.css`. The CSS content includes a comment for the HDI Prediction App, followed by CSS variables for colors, glass background, glass border, glass shadow, text colors, and card radius. It also includes a reset section for margin, padding, and box-sizing, and a smooth scroll behavior for the html element.

```

1 |/* ===== HDI Prediction App - Premium UI/UX Design ===== */
2 |
3 |/* ----- CSS Variables ----- */
4 |
5 |:root {
6 |  --primary: #1e3c72;
7 |  --primary-light: #2a5298;
8 |  --accent: #00d4ff;
9 |  --accent-glow: rgba(0, 212, 255, 0.4);
10 |  --glass-bg: rgba(255, 255, 255, 0.08);
11 |  --glass-border: rgba(255, 255, 255, 0.15);
12 |  --glass-shadow: rgba(0, 0, 0, 0.3);
13 |  --text-primary: #ffffff;
14 |  --text-secondary: rgba(255, 255, 255, 0.75);
15 |  --text-muted: rgba(255, 255, 255, 0.5);
16 |  --card-radius: 20px;
17 |  --transition-smooth: cubic-bezier(0.4, 0, 0.2, 1);
18 |}
19 |
20 |/* ----- Reset ----- */
21 |
22 |* {
23 |  margin: 0;
24 |  padding: 0;
25 |  box-sizing: border-box;
26 |}
27 |
28 |html {
29 |  scroll-behavior: smooth;

```


JavaScript



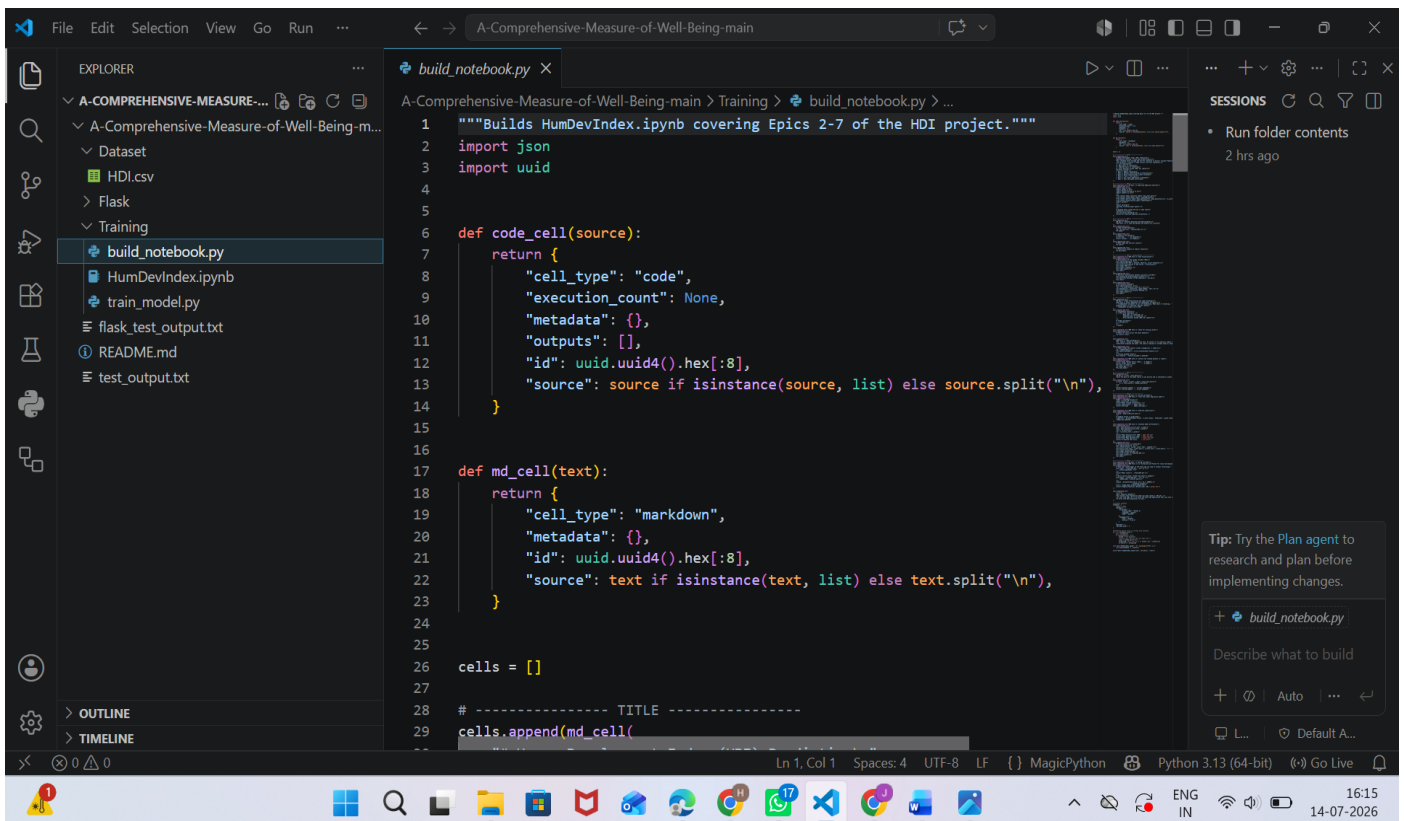
The screenshot shows a VS Code editor window with the file explorer on the left. The active file is `script.js` in the `static` directory. The code is for an HDI Prediction App - Interactive Effects. It includes a `function initParticleBackground()` that sets up a Three.js 3D scene with a perspective camera and a WebGL renderer. The code is as follows:

```

1  /* =====
2  HDI Prediction App - Interactive Effects
3  ===== */
4
5  (function () {
6    "use strict";
7
8    // =====
9    // 1. Three.js 3D Particle Background
10   =====
11   function initParticleBackground() {
12     const canvas = document.getElementById("particle-canvas");
13     if (!canvas) return;
14
15     const scene = new THREE.Scene();
16     const camera = new THREE.PerspectiveCamera(
17       75,
18       window.innerWidth / window.innerHeight,
19       0.1,
20       1000
21     );
22     const renderer = new THREE.WebGLRenderer({
23       canvas,
24       alpha: true,
25       antialias: true,
26     });
27     renderer.setSize(window.innerWidth, window.innerHeight);
28     renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
29   }

```

Backend Implementation Flask(app.py)

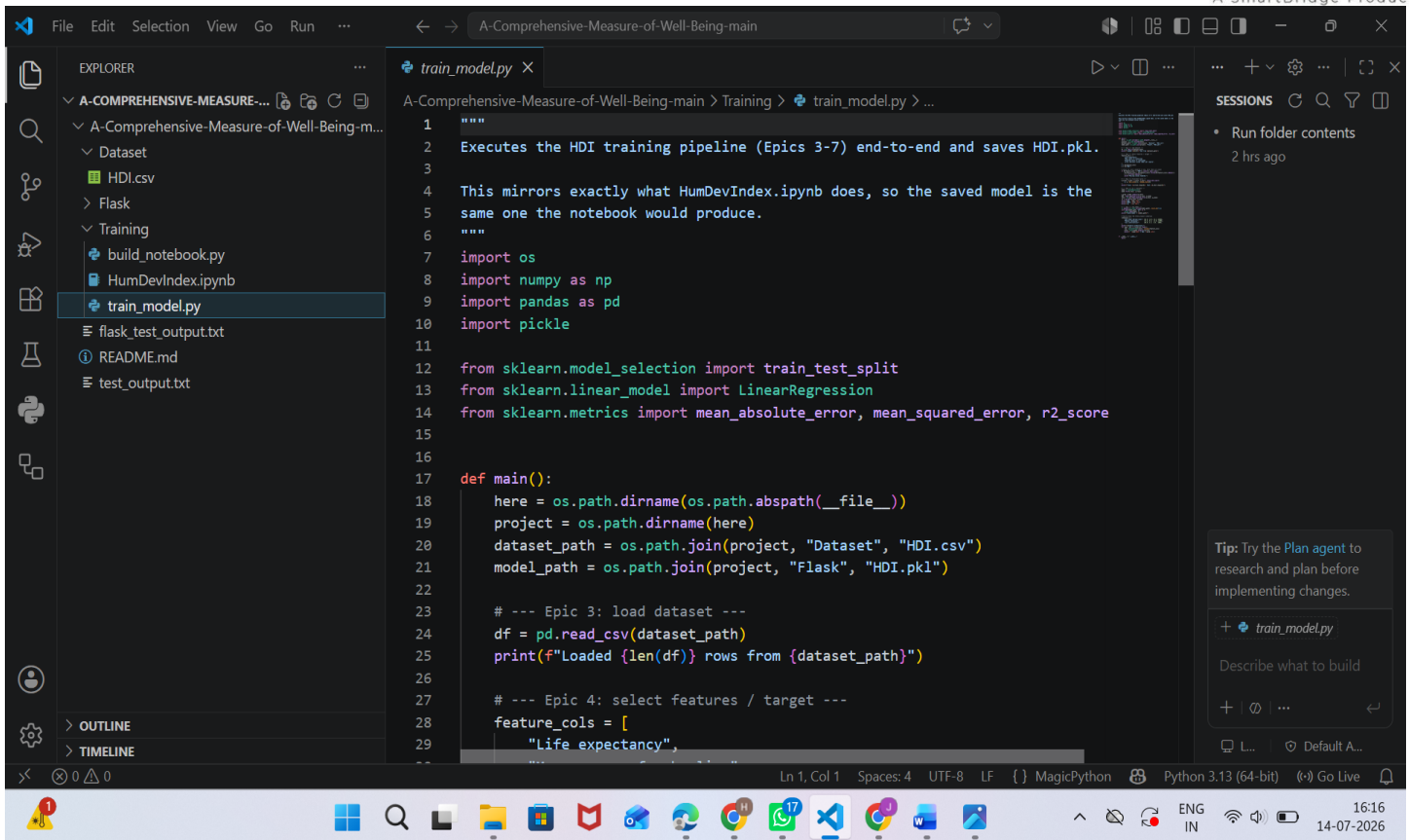


The screenshot shows a VS Code editor window with the file explorer on the left. The active file is `build_notebook.py` in the `Training` directory. The code is for a Flask backend that builds a Jupyter notebook. It includes a `def code_cell(source):` function that returns a dictionary with cell type, execution count, metadata, outputs, id, and source. The code is as follows:

```

1  """Builds HumDevIndex.ipynb covering Epics 2-7 of the HDI project."""
2  import json
3  import uuid
4
5
6  def code_cell(source):
7    return {
8      "cell_type": "code",
9      "execution_count": None,
10     "metadata": {},
11     "outputs": [],
12     "id": uuid.uuid4().hex[:8],
13     "source": source if isinstance(source, list) else source.split("\n"),
14   }
15
16
17  def md_cell(text):
18    return {
19      "cell_type": "markdown",
20      "metadata": {},
21      "id": uuid.uuid4().hex[:8],
22      "source": text if isinstance(text, list) else text.split("\n"),
23    }
24
25
26  cells = []
27
28  # ----- TITLE -----
29  cells.append(md_cell(

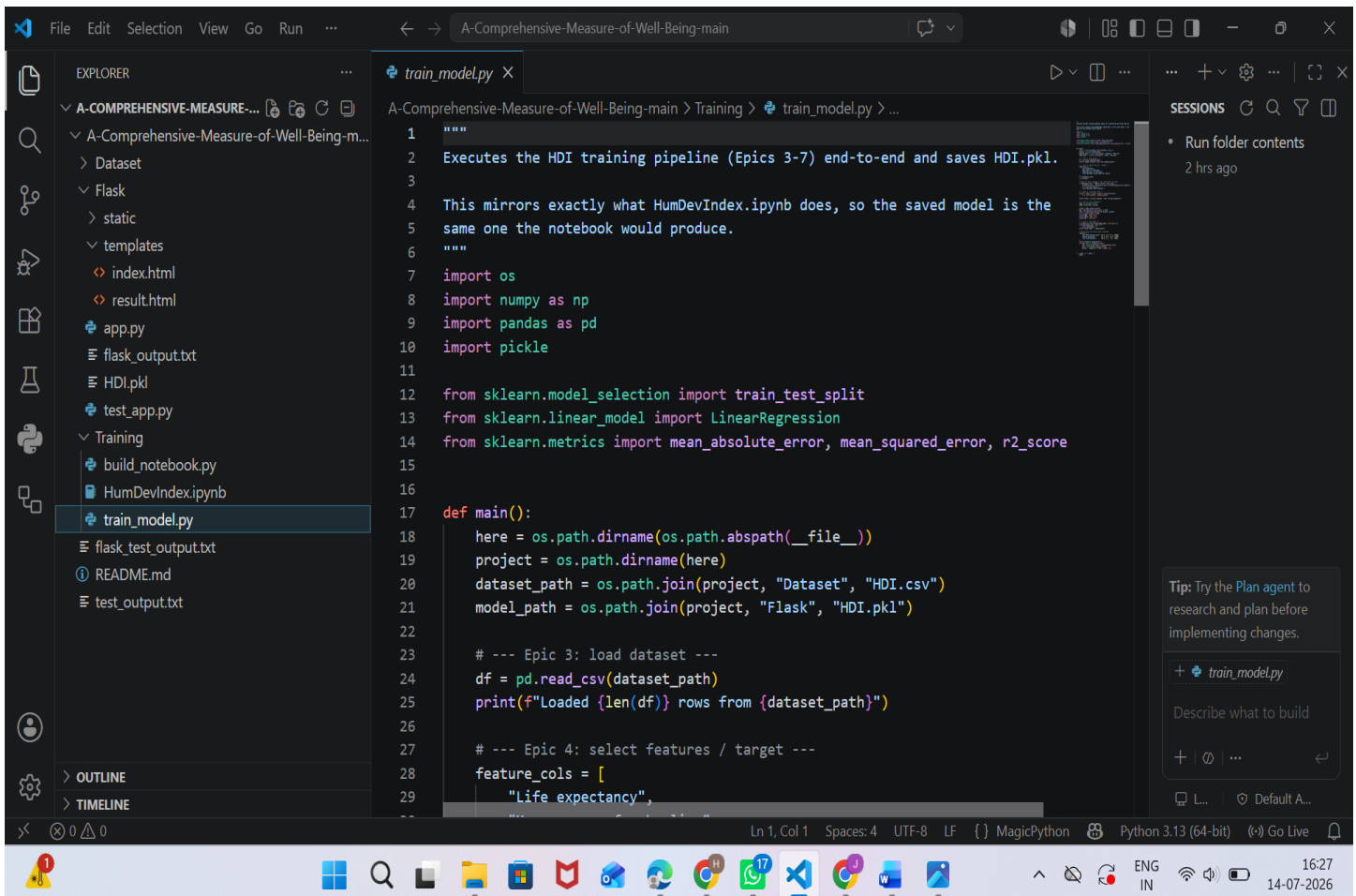
```



The screenshot shows the Visual Studio Code interface with the file explorer on the left, the main editor in the center, and the sessions panel on the right. The file explorer shows the project structure for 'A-COMPREHENSIVE-MEASURE-OF-WELL-BEING-main', including folders for 'Dataset', 'Flask', and 'Training'. The 'train_model.py' file is selected in the 'Training' folder. The main editor displays the code for 'train_model.py', which includes comments about the HDI training pipeline and imports for os, numpy, pandas, and pickle. The code defines a main function that sets up the project path, dataset path, and model path, and then loads the dataset and selects features. The sessions panel on the right shows a session titled 'Run folder contents' with a duration of '2 hrs ago'.

```

1 """
2 Executes the HDI training pipeline (Epics 3-7) end-to-end and saves HDI.pkl.
3
4 This mirrors exactly what HumDevIndex.ipynb does, so the saved model is the
5 same one the notebook would produce.
6 """
7 import os
8 import numpy as np
9 import pandas as pd
10 import pickle
11
12 from sklearn.model_selection import train_test_split
13 from sklearn.linear_model import LinearRegression
14 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
15
16
17 def main():
18     here = os.path.dirname(os.path.abspath(__file__))
19     project = os.path.dirname(here)
20     dataset_path = os.path.join(project, "Dataset", "HDI.csv")
21     model_path = os.path.join(project, "Flask", "HDI.pkl")
22
23     # --- Epic 3: load dataset ---
24     df = pd.read_csv(dataset_path)
25     print(f"Loaded {len(df)} rows from {dataset_path}")
26
27     # --- Epic 4: select features / target ---
28     feature_cols = [
29         "Life expectancy",
30
  
```



This screenshot shows the same Visual Studio Code interface as the first one, but with a different file explorer view. The file explorer now shows the 'A-COMPREHENSIVE-MEASURE-OF-WELL-BEING-main' project structure, including folders for 'Dataset', 'Flask', and 'Training'. The 'train_model.py' file is selected in the 'Training' folder. The main editor displays the code for 'train_model.py', which includes comments about the HDI training pipeline and imports for os, numpy, pandas, and pickle. The code defines a main function that sets up the project path, dataset path, and model path, and then loads the dataset and selects features. The sessions panel on the right shows a session titled 'Run folder contents' with a duration of '2 hrs ago'.

```

1 """
2 Executes the HDI training pipeline (Epics 3-7) end-to-end and saves HDI.pkl.
3
4 This mirrors exactly what HumDevIndex.ipynb does, so the saved model is the
5 same one the notebook would produce.
6 """
7 import os
8 import numpy as np
9 import pandas as pd
10 import pickle
11
12 from sklearn.model_selection import train_test_split
13 from sklearn.linear_model import LinearRegression
14 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
15
16
17 def main():
18     here = os.path.dirname(os.path.abspath(__file__))
19     project = os.path.dirname(here)
20     dataset_path = os.path.join(project, "Dataset", "HDI.csv")
21     model_path = os.path.join(project, "Flask", "HDI.pkl")
22
23     # --- Epic 3: load dataset ---
24     df = pd.read_csv(dataset_path)
25     print(f"Loaded {len(df)} rows from {dataset_path}")
26
27     # --- Epic 4: select features / target ---
28     feature_cols = [
29         "Life expectancy",
30
  
```

Conclusion and Future Scope

Conclusion

The **Human Development Index (HDI) Prediction** project successfully demonstrates how Machine Learning can be used to predict a country's level of human development based on important socio-economic indicators such as **Life Expectancy, Mean Years of Schooling, Expected Years of Schooling, and Gross National Income (GNI) per Capita**. A **Linear Regression** model was developed, trained, and evaluated to generate accurate HDI predictions, and the model was deployed through a **Flask-based web application** that allows users to obtain real-time predictions by entering the required input values. The project provides a simple, reliable, and user-friendly solution for understanding the relationship between health, education, income, and human development. Overall, this system demonstrates the practical application of Machine Learning in socio-economic analysis and serves as a useful tool for researchers, students, policymakers, and organizations interested in evaluating and improving human development.

Future Scope

The **Human Development Index (HDI) Prediction** project has significant potential for future enhancements. Although the current system predicts HDI accurately using four key socio-economic indicators, its performance and usability can be further improved by incorporating larger datasets, additional development indicators, and advanced Machine Learning techniques.

- Integrate real-time data from **UNDP** and other trusted sources.
- Include additional socio-economic indicators such as literacy rate, poverty rate, unemployment rate, healthcare expenditure, and internet access.
- Implement advanced Machine Learning algorithms such as **Random Forest, Decision Tree, XGBoost, and Neural Networks**.
- Compare the performance of multiple prediction models to improve accuracy.
- Develop interactive dashboards with charts and graphical visualizations.
- Enable country-wise comparison and trend analysis over multiple years.
- Deploy the application on cloud platforms for global accessibility.
- Develop a mobile application for easier access on smartphones.
- Add data export features to download prediction reports in PDF or Excel format.
- Improve the user interface with enhanced design and better user experience.
- Support multilingual interfaces to make the application accessible to users worldwide.
- Incorporate AI-based recommendations to help improve a country's human development indicators.

